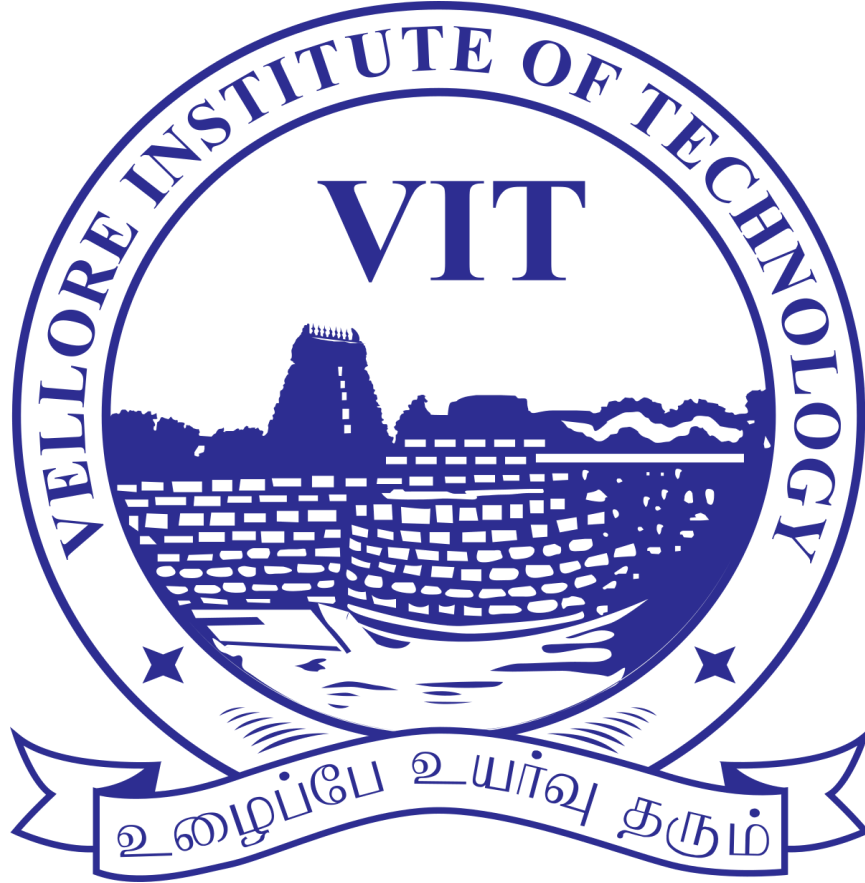


Review 1



Name: Yashita Puri

Registration Number: 23BCI0166

Branch: CSE Information Security

Subject: Software Engineering

Faculty: Prof. Shobha Rekh

Project Name: Intrudex - Smart Intrusion Detection and Alert Engine

Introduction

Traditional authentication systems use static methods like passwords, PINs, or biometric verification. But once an attacker gets hold of these credentials, it's hard for the system to tell if the current user is the real owner of the account. This is a big security hole in modern systems.

To fix this, this project suggests a Behavioral Intrusion Detection System that keeps an eye on user behavior and builds a dynamic user persona.

A user persona is a behavioral profile that shows how a real user interacts with the system.

The system collects multiple behavioral parameters such as:

1. **Mouse interaction patterns**
2. **Keyboard typing dynamics**
3. **System interaction behavior**
4. **Temporal activity patterns**
5. **Network traffic patterns**
6. **Protocol and port usage behavior**

We look at these parameters to generate a behavioral fingerprint for each user. This fingerprint is used to compare real-time user behavior using machine learning and anomaly detection techniques. The system marks the activity as a possible incursion if it sees big changes.

This method is different from standard authentication methods that only check users at login. It allows for continuous authentication, which means that the system can still find unauthorized access even after a successful login.

The system combines behavioral data from many sources to generate a complete user profile. This makes it easier to find intrusions and cuts down on false positives.

Objectives of the Proposed System

The suggested system's key goals are:

1. To create a system for ongoing behavioral authentication that can find unwanted access.
2. To create a multi-modal behavioral persona model that shows how users engage with different parts of the system.
3. To create an adaptive intrusion detection framework that keeps detection accuracy high while lowering the number of false positives.
4. To make security systems more reliable by telling the difference between natural behavioral drift and harmful behavior abnormalities.

Proposed Innovative Components for Patentability

The suggested Behavioral Intrusion Detection System brings new ways to make continuous authentication systems work better. These new ideas are meant to fix the problems with current behavioral biometric systems by making them more adaptable, accurate, and reliable over time.

1. Adaptive Multi-Modal Behavioral Weighting System
2. Engine for Detecting Behavioral Drift

1. Adaptive Multi-Modal Behavioral Weighting Mechanism

Using fixed weights, traditional behavioral authentication systems frequently incorporate more than one biometric signal. But in the actual world, the dependability and availability of behavioral signals might change a lot based on what the user is doing, the system context, and the conditions in the environment.

To fix this problem, the suggested system adds an Adaptive Behavioral Weighting Mechanism that changes the importance of each behavioral signal in real time when it makes the user's persona score.

The system gathers behavioral data from a number of places, such as:

- Mouse interaction patterns
- Keyboard typing dynamics
- System interaction behavior
- Network activity patterns

To get a behavioral confidence score, each behavioral signal is looked at on its own.

Then, these ratings are merged using weights that change over time to get an overall User Persona Score.

Persona Score Calculation

Persona Score = (Wm × MouseScore) + (Wk × KeyboardScore) + (Ws × SystemInteractionScore) + (Wn × NetworkScore)

Wm = Weight assigned to mouse behavior signals

Wk = Weight assigned to keyboard behavior signals

Ws = Weight assigned to system interaction behavior

Wn = Weight assigned to network behavior signals

Weight Adaptation Factors

The system adjusts weights according to several contextual parameters:

- Session context
- Time of day and temporal activity patterns
- Data availability from different behavioral sensors
- Historical reliability of behavioral signals
- Recent anomaly levels detected in each signal type

Example Scenario:

If there isn't much typing going on during a session: Keyboard behavior becomes less reliable. The system automatically reduces keyboard weight.

Similarly, if network behavior shows unusual traffic patterns: The system adds weight to the network.

This adaptive approach makes sure that the intrusion detection algorithm always uses the most trustworthy behavioral signals.

Potential Patent Claim

A system that dynamically adjusts behavioral biometric weights in real time to generate an adaptive user persona score for continuous intrusion detection.

2. Behavioral Drift Detection Engine

User behavior changes over time because of changes in their habits, devices, schedules, and work settings.
Traditional behavioral authentication methods frequently do not differentiate between legitimate behavioral evolution and malicious behavioral oddities.

To solve this problem, the suggested system adds a Behavioral Drift Detection Engine that keeps an eye on changes in the user's behavioral persona and puts them into distinct groups based on how they depart from the norm.

Model of Behavioral State

The technology simulates user activity through three separate states:

Normal Behavior -> Gradual Drift (Learning Phase) -> Anomaly (Possible Intrusion)

Normal Behavior

This state shows patterns of behavior that are quite similar to the user's stored behavioral profile. Small changes within predicted limits are seen as normal.

Gradual Drift (Phase of Learning)

In many real-life situations, how people act may vary over time. Some examples are:
Changing to a new keyboard or device, changing work hours, using different programs, etc.

The system keeps track of how quickly behavior changes from session to session. If behavioral changes happen slowly over a long period of time, the system sees this as natural behavioral drift instead of bad behavior.
The system modifies the user's behavioral persona throughout this phase by employing incremental learning methods.

Anomaly (Possible Intrusion)

The system sees rapid and big changes in the established behavioral identity as a possible incursion.
Some examples are:
A very new pace for typing, patterns of access to applications that are not normal, unusual behavior of network traffic, access from periods of extraordinary activity.

In certain situations, the system sends out intrusion alarms and turns on security response systems.

Differentiating Drift and Intrusion

The Behavioral Drift Engine looks at:

- How fast behavior changes
- Size of the difference
- Reliability of new patterns
- Cross-signal validation from different behavioral sources

This lets the system tell the difference between: Natural behavioral evolution and Malicious user impersonation.

Potential Patent Claim

A method for distinguishing behavioral drift from malicious behavioral deviation in continuous authentication systems using multi-session behavioral change analysis.

Functional Requirements

User Management

1. The system must allow users to authenticate using standard login credentials.
2. The system must maintain behavioral profiles for each registered user.

Behavioral Data Collection

1. The system must collect mouse behavior metrics such as cursor speed, drag duration, and click patterns.
2. The system must capture keyboard typing metrics including typing speed, dwell time, and flight time.
3. The system must monitor system interaction behavior such as application usage and window switching.
4. The system must record temporal activity patterns including active hours and session length.

Network Monitoring

1. The system must monitor network traffic patterns including request rate and burst traffic detection.
2. The system must track protocol usage and detect suspicious activities such as port scanning.

Feature Extraction

1. The system must process raw behavioral data to extract meaningful features.
2. The system must generate behavioral fingerprints for each user.

Intrusion Detection

1. The system must compare real-time behavior against stored user profiles.
2. The system must detect anomalies based on behavioral deviations.

Alert & Response System

1. The system must generate alerts when suspicious behavior is detected.
2. The system must automatically lock the system when intrusion risk exceeds a threshold.
3. The system must allow user verification via secondary authentication.

Reporting & Analytics

1. The system must display behavior analytics on a dashboard.
2. The system must maintain logs of detected anomalies and alerts.

Non - Functional Requirements

Performance

1. The system should detect anomalies in near real-time.
2. Behavioral analysis should not significantly slow down system performance.

Scalability

1. The system should support multiple users simultaneously.
2. The system architecture should scale for enterprise environments.

Security

- 1. All collected behavioral data must be securely stored.
- 2. Communication between system modules must be encrypted.

Accuracy

- 1. The system should minimize false positives.
- 2. The system should continuously improve detection accuracy through model retraining.

Reliability

- 1. The system should operate continuously without interruption.
- 2. Detection services should remain functional and available even during high traffic loads.

Privacy

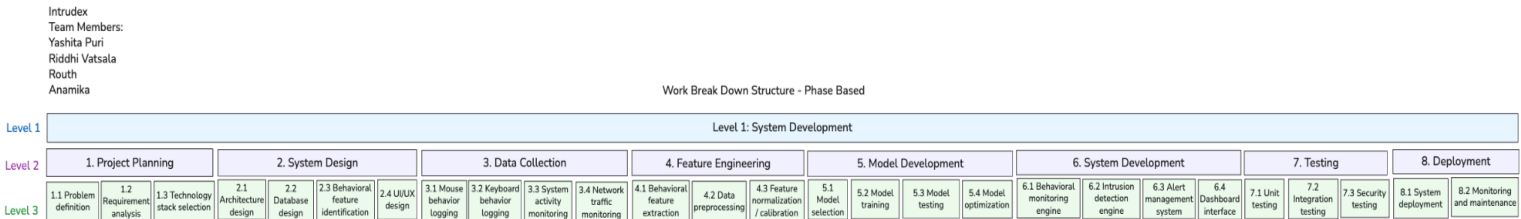
- 1. Behavioral data should be anonymized and then shared.
- 2. User data collection should comply with data protection policies.

Maintainability

- 1. The system should support easy model updates and feature improvements.
- 2. Modular architecture should allow new behavioral features to be added.

Work Breakdown Structures

Phase Based



Phases:

1. Project Planning

Before development starts, this phase is all about figuring out the project's goals, scope, and whether or not it can be done. The team figures out what the problem is, what the system needs, and what technologies to use to make sure the project can be done and grown.

- 1.1 Problem Definition:** Figuring out what the major problem the system is trying to tackle and what the project's goals are.
- 1.2 Requirement Analysis:** Getting and writing down both functional and non-functional needs for the system.
- 1.3 Technology Stack Selection:** Picking the right programming languages, frameworks, databases, and tools for development.

2. System Design

During this phase, the system's overall design and layout are planned. It explains how different parts will work together, how data will be kept, and how users will use the system.

- 2.1 Architecture Design:** Describing how the system will be set up as a whole, including its components and how they will work together.
- 2.2 Database Design:** Figuring out the best way to store, organize, and get to data.
- 2.3 Behavioral Feature Identification:** Finding behaviors or patterns that show normal or suspicious activity on the system.
- 2.4 UI/UX Design:** Making dashboards and system interactions easy to use for everyone.

3. Data Collection

This step is about collecting the information needed to train and test the system's detection methods. To gather useful behavioral data, different sources of system activity are watched.

- 3.1 Mouse Behavior Logging:** Keeping track of how the mouse moves and interacts with other things.
- 3.2 Keyboard Behavior Logging:** Keeping track of things like how fast you type and how many keystrokes you make.
- 3.3 System Activity Monitoring:** Keeping an eye on how processes and the system are being used.
- 3.4 Network Traffic Monitoring:** Recording network activity to find strange patterns in how people talk to each other.

Work Break Down Structure - Phase Based

Level 1: System Development																										
1. Project Planning			2. System Design				3. Data Collection				4. Feature Engineering			5. Model Development				6. System Development				7. Testing			8. Deployment	
1.1 Problem definition	1.2 Requirement analysis	1.3 Technology stack selection	2.1 Architecture design	2.2 Database design	2.3 Behavioral feature identification	2.4 UI/UX design	3.1 Mouse behavior logging	3.2 Keyboard behavior logging	3.3 System activity monitoring	3.4 Network traffic monitoring	4.1 Behavioral feature extraction	4.2 Data preprocessing	4.3 Feature normalization / calibration	5.1 Model selection	5.2 Model training	5.3 Model testing	5.4 Model optimization	6.1 Behavioral monitoring engine	6.2 Intrusion detection engine	6.3 Alert management system	6.4 Dashboard interface	7.1 Unit testing	7.2 Integration testing	7.3 Security testing	8.1 System deployment	8.2 Monitoring and maintenance

1. Project Planning			2. System Design				3. Data Collection				4. Feature Engineering			5. Model Development				6. System Development				7. Testing			8. Deployment	
1.1 Problem definition	1.2 Requirement analysis	1.3 Technology stack selection	2.1 Architecture design	2.2 Database design	2.3 Behavioral feature identification	2.4 UI/UX design	3.1 Mouse behavior logging	3.2 Keyboard behavior logging	3.3 System activity monitoring	3.4 Network traffic monitoring	4.1 Behavioral feature extraction	4.2 Data preprocessing	4.3 Feature normalization / calibration	5.1 Model selection	5.2 Model training	5.3 Model testing	5.4 Model optimization	6.1 Behavioral monitoring engine	6.2 Intrusion detection engine	6.3 Alert management system	6.4 Dashboard interface	7.1 Unit testing	7.2 Integration testing	7.3 Security testing	8.1 System deployment	8.2 Monitoring and maintenance

4. Feature Engineering

Feature engineering changes raw data into inputs that machine learning models can understand. This step makes the data better and more valuable for making accurate forecasts.

- 4.1 Behavioral Feature Extraction:** Getting important behavioral traits from the data you collected.
- 4.2 Data Preprocessing:** Getting the data ready for model training by cleaning, filtering, and preparing it.
- 4.3 Feature Normalization/Calibration:** Making features the same so that algorithms can compare and use them correctly.

5. Model Development

This phase is all about creating machine learning models that can look at how users behave and find problems. The model is trained, tested, and improved to make sure it is very accurate and dependable.

- 5.1 Model Selection:** Picking the right machine learning methods for finding things.
- 5.2 Model Training:** Using the datasets that have been put up to train the model.
- 5.3 Model Testing:** Using test data to see how well the model works.

5.4 Model Optimization: Making the model better by changing its settings and making it more accurate.

6. System Development

In this stage, the real application is made by putting detection models along with system parts. This includes developing user dashboards, alarm systems, and monitoring modules.

- 6.1 Behavioral Monitoring Engine:** Setting up the system that keeps an eye on what users do all the time.
- 6.2 Intrusion Detection Engine:** Making the part that looks at behavior and finds problems.
- 6.3 Alert Management System:** Making a system that sends out notifications when it sees something suspicious.
- 6.4 Dashboard Interface:** Creating a visual interface that lets admins keep an eye on what's going on in the system.

7. Testing

Before putting the system into use, testing makes sure it operates accurately, safely, and quickly. Testing at different levels makes sure that all the parts work together correctly.

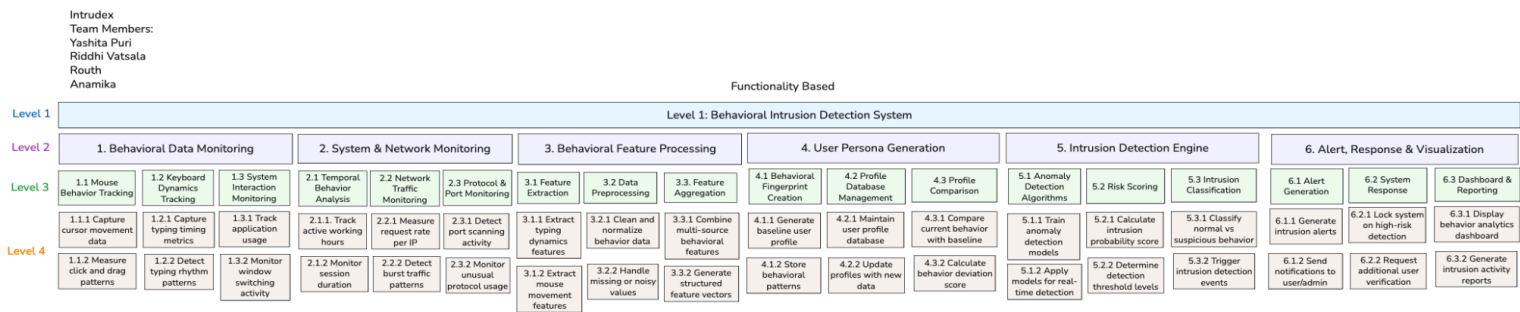
- 7.1 Unit Testing:** Checking each module on its own to make sure it works well.
- 7.2 Integration Testing:** Checking to see if the individual parts function together as they should.
- 7.3 Security Testing:** Looking for weaknesses in the system and make sure it can withstand attacks.

8. Deployment

Deployment is the last step, when the system is made available for usage in the real world. After deployment, the system is always watched over and cared for to make sure it stays stable and works well.

- 8.1 System Deployment:** Setting up the system on the server or environment where it will be used.
- 8.2 Monitoring and Maintenance:** Keeping an eye on how well the system is working and correct problems when they come up.

Functionality Based



Major Modules:

1. Behavioral Data Monitoring

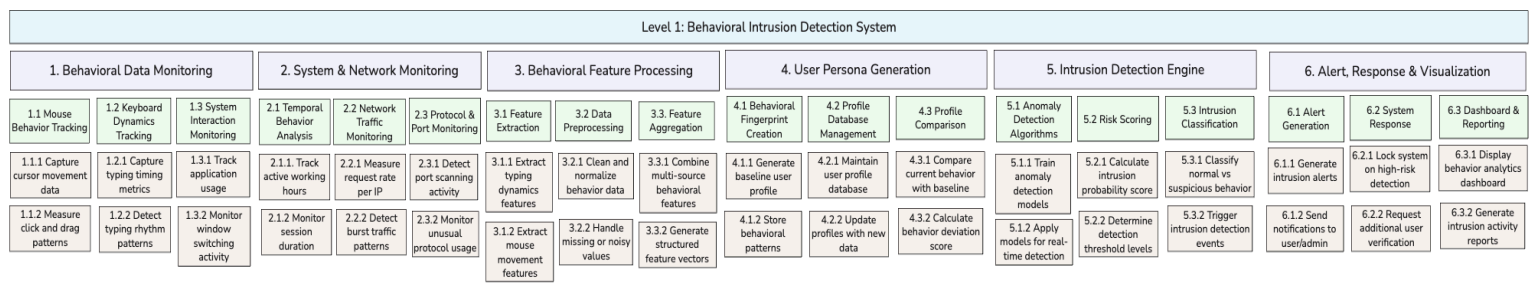
This module is in charge of gathering real-time behavioral data from users as they use the system. The system collects raw behavioral data by keeping track of things like mouse movements, typing habits, and application usage. This data helps find patterns in normal user behavior.

- 1.1 Mouse Behavior Tracking: Records and analyzes how the pointer moves, how often it clicks, and how it navigates.
- 1.2 Keyboard Dynamics Tracking: Keeps track of things like typing speed, keystroke timing, and typing rhythm.
- 1.3 System Interaction Monitoring: Keeps an eye on how users interact with apps, windows, and system resources.

2. System & Network Monitoring

This module keeps an eye on how the system is being used and what is happening on the network to find unusual access or suspicious operations. It looks at user sessions, network requests, and communication patterns to find possible strange activity.

- 2.1 Temporal Behavior Analysis: Looks at how users act across time, such as when they log in, how long they work, and how often they have sessions.
- 2.2 Network Traffic Monitoring: Keeps an eye on network requests, traffic rates, and how devices talk to each other.
- 2.3 Protocol and Port Monitoring: This feature finds strange use of network ports and communication protocols.



3. Behavioral Feature Processing

This module takes raw behavioral data from monitoring components and turns it into useful features for study. It makes sure that the data is clean, organized, and ready for machine learning algorithms that are used to find intrusions.

- 3.1 Feature Extraction: This takes essential behavioral traits from raw activity data that has been obtained.
- 3.2 Data Preprocessing: This step cleans, normalizes, and gets behavioral data ready by dealing with noise and missing information.
- 3.3 Feature Aggregation: Puts together features from several monitoring sources into one dataset.

4. User Persona Generation

This module builds a behavioral profile for each user based on how they have interacted with the system in the past. The created persona serves as a baseline model that shows regular behavior, which lets the system spot changes in future actions.

- 4.1 Behavioral Fingerprint Creation: This makes a unique behavioral signature that shows how a normal user acts.
- 4.2 Profile Database Management: Keeps track of and updates users' behavioral profiles in a single database.
- 4.3 Profile Comparison: This feature compares real-time behavioral data with saved profiles to find differences.

5. Intrusion Detection Engine

This module is the main part of the system that analyzes data to find suspicious behavior. It uses algorithms for finding anomalies, figures out risk scores, and sorts behavioral aberrations to figure out if an intrusion has happened.

5.1 Anomaly Detection Algorithms: These use machine learning models to find unusual behavior patterns.

5.2 Risk Scoring: Figures out a risk score that shows how likely it is that something suspicious is going on.

5.3 Intrusion Classification: Sorts unusual activities into normal or malignant categories.

6. Alert, Response & Visualization

This module controls how the system responds to threats and how security information is shown to administrators. It sends out notifications, starts automated replies, and gives you dashboards to keep an eye on system activity and security occurrences.

6.1 Alert Generation: Sends out alerts when strange or suspicious conduct is found.

6.2 System Response: Starts security measures such as alarms, requests for verification, or limiting access.

6.3 Dashboard and Reporting: A monitoring dashboard shows behavioral analytics, alarms, and system reports.

Class Diagram

Workflow:

1. Register User

The real user installs the Intrudex system and sets up their account and the device that is allowed to use it. When a user signs up, the system saves basic information about them and ties the device ID to make sure that future behavior tracking is linked to the right user.

2. Collect Behavioral Data

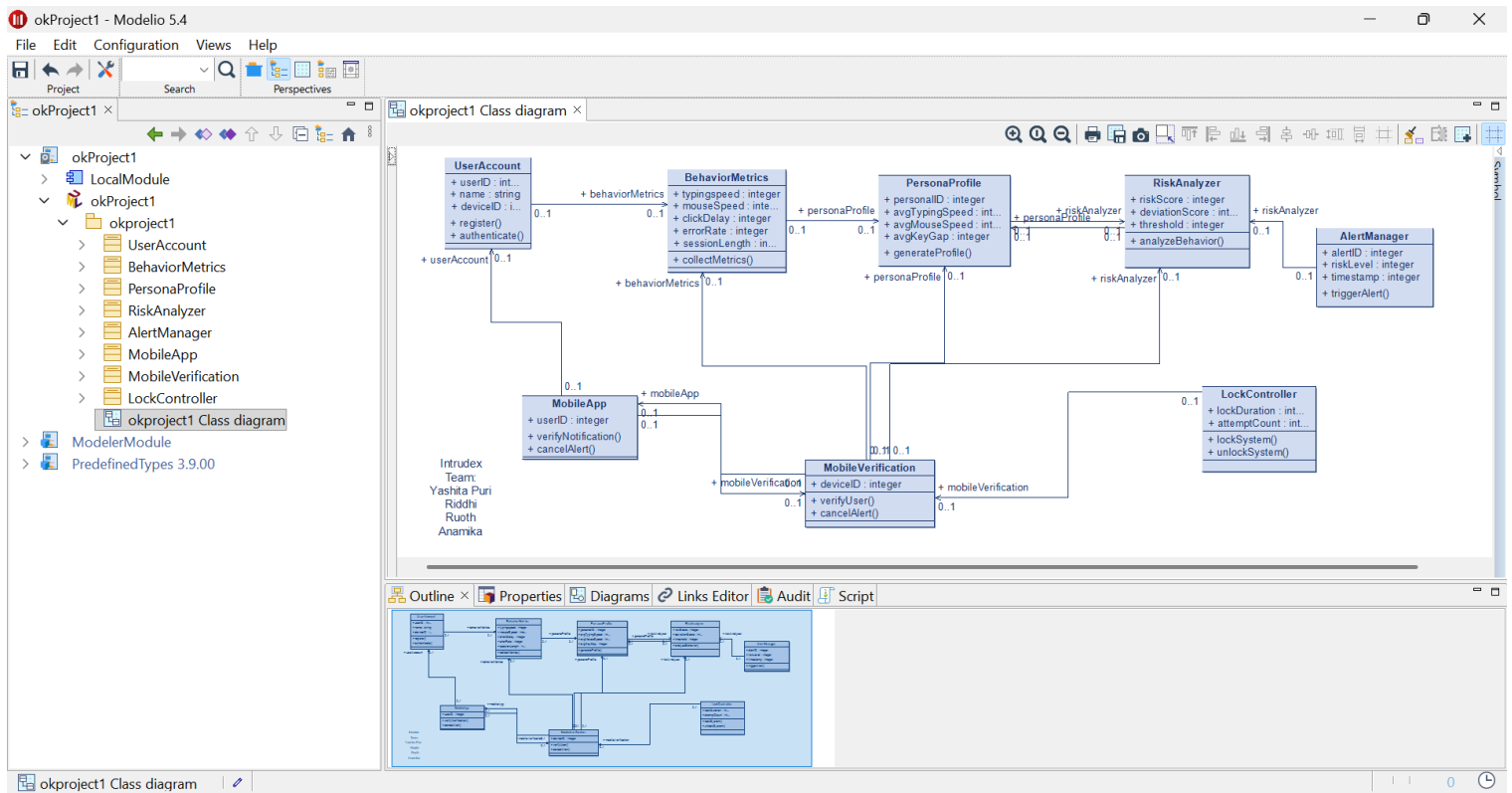
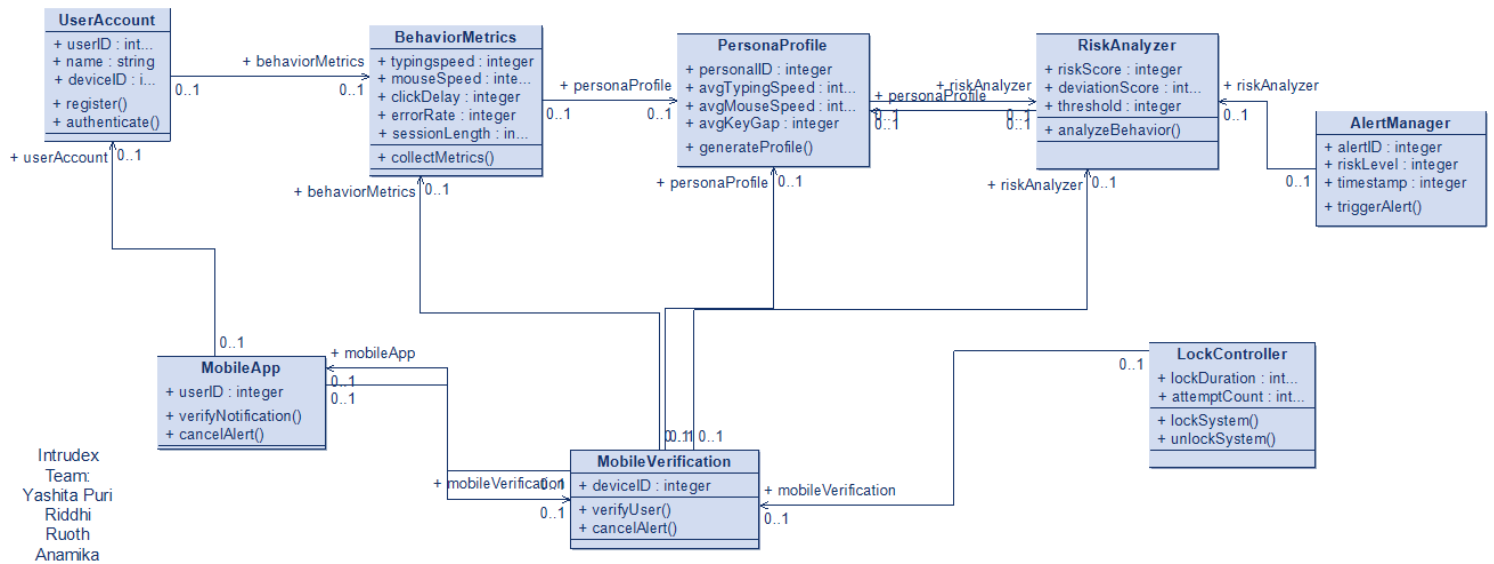
While the user is using the gadget, the system captures behavioral metrics all the time during the first 15 days of training. These measures include typing speed, mouse movement patterns, click latency, and session time, which assist represent the way a real user interacts with the computer.

3. Generate User Persona

The algorithm looks at the parameters and makes a behavioral persona after it has enough behavioral data. This persona is a basic behavioral model that shows how a real user would normally interact with the system.

4. Monitor User Activity

After the training phase is over, the system starts to monitor things in real time. It keeps an eye on how users type, move their mouse, and use the system during every session.



5. Analyze Behavior and Detect Anomalies

The system checks the current behavior data against the stored persona profile to find any differences. The system figures out a risk score to see how likely an incursion is if the behavior is very different from the baseline.

6. Trigger Intrusion Alert

The technology flags the action as possibly suspicious when the estimated risk score goes above a certain level. It delivers an alarm about an intrusion, shows a warning message on the system, and sends a notification to the registered mobile app.

7. Mobile Verification

The consumer gets the notice on their mobile app and can check to see if the activity is real. The system sees the action as a false alarm and cancels the alert if the user verifies it.

8. Lock System

The system's security response mechanism kicks in if the suspicious activity continues or the user doesn't prove their identity. The device is momentarily locked, and the lock time gets longer each time someone tries to break in to keep people from getting in without permission.

Classes and their Relationships in Intrudex

User Account

The UserAccount class is for people who have signed up for the system. It keeps track of user information including their ID, name, and device ID, and it has tools for signing up and logging in.

BehaviorMetrics

The BehaviorMetrics class gathers information about how people interact with computers, like how fast they type, how fast they move their mouse, how long it takes them to click, how many errors they make, and how long their sessions last. These metrics are constantly being collected while users are active, and they are used to analyze behavior.

PersonaProfile

The PersonaProfile class keeps track of a user's basic behavior profile. It keeps track of average values for things like typing speed, mouse speed, and key gap intervals that show how a normal user would act.

RiskAnalyzer

The RiskAnalyzer class looks at behavioral deviations by comparing real-time behavioral variables with the stored persona profile. It figures out the risk level and deviation scores depending on set thresholds.

AlertManager

The AlertManager class takes care of notifications that are sent out when questionable behavior is found. It keeps track of alert information including the alert ID, risk level, and time stamp, and it sends out alerts or warning messages.

LockController

The LockController class makes sure that the system responds in a secure way. When it sees high-risk activities, it can briefly lock the device and then unlock it after checking.

MobileVerification

The MobileVerification class adds another layer of security by checking the user's identification through the mobile device they registered. It helps determine if anything that looks strange is real.

Mobile App

The MobileApp class is what the user uses to get security alerts and requests for verification on their phone. It lets consumers verify real activities and turn off warnings when they need to.

Relationships Between Classes

UserAccount to BehaviorMetrics

Because the system keeps track of behavioral data during different sessions, one user account can create many behavioral metric records.

BehaviorMetrics to PersonaProfile

A single persona profile that shows the user's regular behavior is generated and updated by looking at several behavioral metrics at once.

PersonaProfile to RiskAnalyzer

The risk analyzer looks at real-time behavior variables and compares them to the stored persona profile to find differences and figure out risk scores.

RiskAnalyzer to AlertManager

When the risk score goes above the set limit, the risk analyzer sends the information to the alert manager, which sends out an intrusion alarm.

LockController to AlertManager

The alert manager tells the lock controller to temporarily lock the system if the threat level is high.

MobileVerification to MobileApp

The mobile verification module talks to the mobile app to submit requests for verification and get confirmation from the real user.

MobileVerification to LockController

Once the person has been verified, the mobile verification module can tell the lock controller to unlock the system or stop security activities.

MobileApp to UserAccount

The UserAccount class and the MobileApp class are related because the mobile app is tied to the registered user of the system. It uses the UserAccount's user ID to deliver verification messages and let the user confirm or dismiss intrusion alerts.

Lean Canvas

Problem Static logins leave sessions open when users walk away. Unauthorized users can access active sessions. Strong security measures can conflict with user privacy.	Solution Continuous behavioral authentication using typing patterns. Real-time monitoring of keyboard, mouse, and user interactions. Anomaly detection to identify suspicious behavior.	Unique Value Proposition Hardware-free security running silently in the background. Continuously verifies user identity during system use. Protects workstations without interrupting workflow.	Unfair Advantage Hybrid machine learning models for behavioral authentication. Unique behavioral persona profiles for each user. Privacy-friendly monitoring based on interaction patterns.	Customer Segments Organizations requiring secure workstation access. Enterprises protecting sensitive systems. Security teams monitoring insider threats.
Existing Alternatives Hardware-based biometrics like fingerprint systems. Passwords and MFA that only secure login. Traditional IDS with limited behavioral analysis.	Key Metrics FAR and FRR to measure authentication accuracy. Behavioral anomaly detection accuracy.	High Level Concept A behavioral security system that continuously verifies the user during system usage.	Channels Cybersecurity blogs and technical publications. Partnerships with security solution providers.	Early Adopters Organizations with shared workstations. Companies concerned about insider threats.
Cost Structure Development team salaries. Cloud infrastructure and maintenance.			Revenue Streams Subscription model (~₹340 per month). Free trial to encourage adoption.	

Issue

When you log in, traditional systems merely check your identity once. Someone who isn't supposed to be there could get into an active session if a user leaves their workstation unattended. Too many surveillance tools might also make employees worry about their privacy and lower their trust.

Existing Alternatives

Common security measures are physical biometrics like fingerprint or facial recognition, regular passwords with two-factor authentication, and standard intrusion detection systems. But these approaches frequently need expensive technology, just protect the login stage, or don't check users all the time.

Solution

Intrudex employs behavioral authentication to check users' keyboard dynamics throughout the entire session. The technology can find unusual activity and possible unauthorized access in real time by looking at typing patterns and how users interact with each other.

Unique Value Proposition

Intrudex offers a security solution that doesn't require any hardware and protects users' privacy by constantly checking their identity. It runs in the background without making any noise, like a digital guard that keeps systems safe without getting in the way of typical user activities.

Customer Segments

Enterprise security teams, businesses that share workstations, and businesses that handle sensitive data that needs constant and privacy-conscious security monitoring are the main users.

Problem Static logins leave sessions open when users walk away. Unauthorized users can access active sessions. Strong security measures can conflict with user privacy.	Solution Continuous behavioral authentication using typing patterns. Real-time monitoring of keyboard, mouse, and user interactions. Anomaly detection to identify suspicious behavior.	Unique Value Proposition Hardware-free security running silently in the background. Continuously verifies user identity during system use. Protects workstations without interrupting workflow.	Unfair Advantage Hybrid machine learning models for behavioral authentication. Unique behavioral persona profiles for each user. Privacy-friendly monitoring based on interaction patterns.	Customer Segments Organizations requiring secure workstation access. Enterprises protecting sensitive systems. Security teams monitoring insider threats.
Existing Alternatives Hardware-based biometrics like fingerprint systems. Passwords and MFA that only secure login. Traditional IDS with limited behavioral analysis.	Key Metrics FAR and FRR to measure authentication accuracy. Behavioral anomaly detection accuracy.	High Level Concept A behavioral security system that continuously verifies the user during system usage.	Channels Cybersecurity blogs and technical publications. Partnerships with security solution providers.	Early Adopters Organizations with shared workstations. Companies concerned about insider threats.
Cost Structure Development team salaries. Cloud infrastructure and maintenance.			Revenue Streams Subscription model (~₹340 per month). Free trial to encourage adoption.	

Early Adopters

Early adopters may include businesses and organizations that need strong workstation security, especially in places where keeping an eye on user behavior might help find unusual or unsafe behavior.

Channels

Customers can learn about the product through technical blogs, whitepapers on cybersecurity, product demos, and partnerships with companies that offer cybersecurity solutions.

Key Metrics

False Accept Rate (FAR) and False Reject Rate (FRR) are used to check the correctness of authentication and see how well the system works.

Unfair Advantage

Intrudex uses a mix of Random Forest and One-Class SVM machine learning to find unusual behavior patterns with a high degree of accuracy.

Cost Structure

The biggest expenditures are the pay of the development team, the cloud hosting infrastructure, and the actions needed to make sure security is up to code.

Revenue Streams

Intrudex works on a subscription basis, with a monthly fee of ₹340 and a 30-day free trial to get people to sign up.

